

RESEARCH wireguard

RESEARCH wireguard

Revision: 1 **Last modified:** 2026-06-25T12:00:00Z

Web-grounded research into the **WireGuard protocol + its userspace/kernel implementations** — the cryptographic core HelixVPN builds on (helix-wg, [v02 wireguard-core.md]). Backs the UNVERIFIED: markers in that spec doc (wire sizes, crypto set, timers, boringtun API). Every external fact is cited inline by source id; access date **2026-06-25**. All source URLs are listed in the "## Sources verified" footer. Anything a source could not confirm is marked UNVERIFIED: per constitution §11.4.6 — never guessed.

One source was unreachable (honest gap, §11.4.6/ §11.4.99): the formal **WireGuard whitepaper PDF** (wireguard.com/papers/wireguard.pdf) returned raw FlateDecode-compressed binary through the fetch tool, NOT parseable text — see [wg-paper-UNREACHABLE]. Every value that the whitepaper would canonically source was therefore re-confirmed from a *different* authoritative source: the official **Protocol & Cryptography** page [wg-protocol] and the official `messages.h` constants header [wg-messages-h]. No protocol value below rests on the unreachable PDF alone.

1. Protocol shape — Noise IK, 1-RTT

WireGuard's handshake is the **Noise_IK** pattern (more precisely **Noise_IKpsk2** when the optional preshared key is in use, §6): a **1-RTT** key exchange where the Initiator already **K**nows the responder's static public key in advance [wg-protocol]. The initiator sends message 1, the responder replies with message 2, and only then may data flow — and critically "the responder cannot [send data] until receiving an encrypted packet from the initiator for key confirmation" [wg-protocol]. So the responder never speaks first and never initiates a rekey; it can only hint (via a keepalive) that the initiator should rekey.

This is exactly the architecture HelixVPN's helix-wg drives: the responder static pubkey is learned out-of-band from the coordinator's RouteMap (PeerRoute.wg_pubkey) before the handshake — the "K" precondition [v02 wireguard-core §3.1].

2. Cryptographic primitives (the fixed set)

WireGuard fixes one primitive per role — there is no cipher negotiation, no downgrade surface (this is the "cryptographically opinionated" design) [wg-protocol]:

Role	Primitive	Source
Key agreement (ECDH)	Curve25519 (X25519)	[wg-protocol]
AEAD (handshake + data)	ChaCha20Poly1305 , RFC 7539 construction (96-bit nonce = 32-bit zero 64-bit LE counter)	[wg-protocol]
Hashing / keyed hashing / KDF	BLAKE2s (RFC 7693) + HKDF (RFC 5869)	[wg-protocol]
Hashtable keys (index lookup)	SipHash24	[wg-protocol]
Cookie encryption (anti-DoS, §5)	XChaCha20Poly1305 (24-byte random nonce)	[wg-protocol]
Handshake replay (msg 1)	TAI64N 12-byte timestamp	[wg-protocol]

helix-wg's invariant W2 ("crypto parameters fixed, never altered") matches the WireGuard design exactly [v02 wireguard-core §4.1]. SipHash24 and XChaCha20Poly1305 were UNVERIFIED: in the spec doc and are now CONFIRMED from [wg-protocol] — SipHash24 keys the index hashtable; XChaCha20Poly1305 encrypts the cookie.

3. The four message types + exact wire sizes (CONFIRMED)

These sizes were UNVERIFIED: in [v02 wireguard-core §3] (the spec doc flagged the absent research-wireguard.md). They are now **CONFIRMED** against the official protocol page [wg-protocol].

3.1 Message 1 — Handshake Initiation = 148 bytes [wg-protocol]

offset	size	field
0	1	message_type = 1
1	3	reserved_zero = 0x000000
4	4	sender_index (u32 LE)
8	32	unencrypted_ephemeral (Curve25519 ephemeral public)
40	48	encrypted_static (32 B static pub + 16 B Poly1305 tag)
88	28	encrypted_timestamp (12 B TAI64N + 16 B tag)
116	16	mac1
132	16	mac2
= 148 bytes total		

Initiator operations [wg-protocol]: generate ephemeral via DH_GENERATE(); send the ephemeral public in clear; encrypt the static public key under a derived key; encrypt the **12-byte TAI64N timestamp**; compute mac1 = MAC(HASH(LABEL_MAC1 || responder.static_public), msg[0:offsetof(mac1)]); compute mac2 from the last received cookie (or zeros if none, §5).

3.2 Message 2 — Handshake Response = 92 bytes [wg-protocol]

offset	size	field
0	1	message_type = 2
1	3	reserved_zero
4	4	sender_index (responder's index)
8	4	receiver_index (echoes initiator's sender_index)
12	32	unencrypted_ephemeral
44	16	encrypted_nothing (empty payload + 16 B tag – key confirmation)
60	16	mac1
76	16	mac2
= 92 bytes total		

Responder operations [wg-protocol]: generate its ephemeral; perform three DH operations (ephemeral↔ephemeral, ephemeral↔initiator-static, then **mix in the preshared key**, §6); encrypt an empty plaintext (authentication-only, confirms the key schedule).

3.3 Message 3 — Cookie Reply = 64 bytes [wg-protocol]

offset	size	field
0	1	message_type = 3

1	3	reserved_zero	
4	4	receiver_index	
8	24	nonce	(random, for XChaCha20Poly1305)
32	32	encrypted_cookie	(16 B cookie + 16 B tag)
			= 64 bytes total

Sent only when the responder is under load and mac2 is invalid/absent (§5). "Cookies expire after two minutes" [wg-protocol] — confirmed by COOKIE_SECRET_MAX_AGE = 2 * 60 [wg-messages-h].

3.4 Message 4 — Transport Data (variable) [wg-protocol]

offset	size	field	
0	1	message_type	= 4
1	3	reserved_zero	
4	4	receiver_index	(demux key – the local index the peer was told)
8	8	counter	(u64 LE nonce; monotonic; "Nonces are never reused")
16	N	encrypted_encapsulated_packet	(ChaCha20Poly1305; inner pkt padded to 16 B + 16 B tag)

The fixed **16-byte data header + 16-byte Poly1305 tag = 32-byte WireGuard overhead** is exactly what the MTU budget (§7 below) subtracts. The inner packet is zero-padded to a multiple of 16 bytes before AEAD [wg-protocol]. This confirms [v02 wireguard-core §3.4].

4. Key schedule + data keys (CONFIRMED chaining)

After the handshake, the two directional transport keys are derived from the final chaining key via HKDF-style BLAKE2s HMAC [wg-protocol]:

```
temp1 = HMAC(chaining_key, [empty])
temp2 = HMAC(temp1, 0x1)
temp3 = HMAC(temp1, temp2 || 0x2)
sending_key = temp2
receiving_key = temp3
```

The counter starts at 0; "all previous chaining keys, ephemeral keys, and hashes are zeroed out" once data keys are derived [wg-protocol]. This grounds the spec doc's UNVERIFIED: "exact chaining" note [v02 wireguard-core §4.2].

5. Anti-DoS: MAC1, MAC2, cookies (CONFIRMED)

WireGuard layers two MACs onto every handshake message [wg-protocol]:

- **MAC1** = `MAC(HASH(LABEL_MAC1 || receiver.static_public), msg[0:offsetof(mac1)])` — a keyed-BLAKE2s MAC over the static public key. It is the **cheap first gate**: a message with a bad MAC1 is dropped *before* any expensive Curve25519 work. An attacker who does not know the responder's static public key cannot forge MAC1.
- **MAC2** = `MAC(cookie, msg[0:offsetof(mac2)])`, valid only when the initiator holds a current cookie. Under load the responder rejects handshakes with an invalid MAC2 and replies with a **Cookie Reply (msg 3)**.
- **Cookie** = `MAC(responder.changing_secret_every_two_minutes, initiator.ip_address)` — proves the initiator owns its claimed source IP and enables per-source rate limiting [wg-protocol]. The secret rotates every two minutes (`COOKIE_SECRET_MAX_AGE = 2*60` [wg-messages-h]).

This is the grounded architecture behind helix-wg's BadMac1/BadMac2 errors and HandshakeRateLimiter [v02 wireguard-core §10, §14.3]. boringtun exposes a `rate_limiter` submodule that implements this cookie path (§9).

6. The preshared key (PSK) — the post-quantum injection point (CONFIRMED)

WireGuard has an **optional 32-byte symmetric preshared key**. "When pre-shared key mode is *not* in use, the pre-shared key value used below is assumed to be an all-zero string of 32 bytes" [wg-protocol] — i.e. the PSK slot always exists; it is simply zero when unused. In the Noise_IKpsk2 variant the PSK is mixed into the chaining key during response generation: `temp = HMAC(responder.chaining_key, preshared_key)` [wg-protocol]. Because it enters the **symmetric** key schedule, the tunnel stays secure even if the X25519 ECDH is later broken — this is precisely the property a post-quantum layer exploits.

Why this is HelixVPN's PQ seam (cross-referenced, grounded). Mullvad's quantum-resistant tunnels do exactly this: establish a vanilla WireGuard tunnel, run a post-quantum KEM exchange (ML-KEM-1024 / Classic McEliece, hybrid by default) *over* it, then install the negotiated shared secret as WireGuard's

PSK [research-mullvad §4]. The WireGuard protocol surface needs **zero change** — only the PSK field is filled. `helix-wg`'s only surface is `PeerConfig.psk: Option<Psk>`

- the §4 key-schedule mix-in [v02 wireguard-core §4.4]. The UAPI exposes the PSK as the `preshared_key` config key (hex; all-zero to remove) [wg-xplatform].

UNVERIFIED: the exact PQ-PSK message sequence (KEM exchange framing) — this is a Mullvad-specific protocol on top of WireGuard, deliberately high-level in public docs [research-mullvad §"Honest gaps"]; it is HelixVPN security-doc design work, NOT part of the WireGuard protocol itself.

7. AllowedIPs — Cryptokey Routing (CONFIRMED semantics)

AllowedIPs is WireGuard's **Cryptokey Routing** table and is dual-purpose [wg-xplatform]:

- **Outbound (routing)**: an outgoing plaintext packet's **destination** IP selects which peer (hence which key) to encrypt for — longest-prefix match across all peers' AllowedIPs.
- **Inbound (access control)**: a packet decrypted from a given peer is only accepted if its inner **source** IP falls within *that* peer's AllowedIPs; otherwise it is dropped. A peer cannot inject a source it was not granted.

The UAPI configures this per peer with `allowed_ip` (CIDR), `replace_allowed_ips`, `public_key`, `endpoint`, `persistent_keepalive_interval`, and `preshared_key` [wg-xplatform]. This is the grounding for `helix-wg`'s AllowedIpsTrie and its W4 default-deny invariant [v02 wireguard-core §5]. The kernel implementation backs it with a Patricia/radix trie keyed on prefix → peer; the userspace UAPI exposes the same semantics over a UNIX-socket text protocol (`set=1 ...`, `get=1`) [wg-xplatform].

8. MTU — why 1420, and the overhead math (CONFIRMED)

WireGuard's default tunnel MTU is **1420** over a standard 1500-byte Ethernet path [wg-mtu]. The overhead breakdown [wg-mtu]:

- **IPv4 outer transport**: 20 B IPv4 header + 8 B UDP header + **32 B WireGuard overhead** (16 B data header + 16 B Poly1305 tag, §3.4) = **60 B** → $1500 - 60 = \mathbf{1440}$, but

WireGuard defaults to **1420** so the *same* value also covers the IPv6 case.

- **IPv6 outer transport:** 40 B IPv6 header + 8 B UDP + 32 B WireGuard = **80 B** → 1500 – 80 = **1420**.

So 1420 is the floor that works whether the outer transport is IPv4 or IPv6 [wg-mtu]. The practical rule: "set the WireGuard interface MTU 60 bytes smaller than the narrowest link for IPv4, 80 bytes smaller for IPv6" [wg-mtu]. For mobile clients over heterogeneous paths, **1280** (the IPv6 minimum MTU) is the robust choice that survives almost any carrier [wg-mtu] — which is exactly why HelixVPN's masque-h3 transport targets a 1280 inner MTU [v02 wireguard-core §7.1].

This CONFIRMS the spec doc's plain-udp 1420 figure and resolves its UNVERIFIED: "does 1420 fold WG or IP overhead" note [v02 wireguard-core §7.2]: the 32-byte WireGuard overhead **is** included in the 60/80-byte subtraction, so a 1420 inner MTU already accounts for the WireGuard data header + tag.

9. Timer / rekey / keepalive constants (CONFIRMED exact values)

These were UNVERIFIED: in [v02 wireguard-core §3.6/§8.2]. Now **CONFIRMED** from the official messages.h header [wg-messages-h] and the protocol page [wg-protocol] / wireguard-go timers [wg-timers-search]:

Constant	Value	Meaning
REKEY_AFTER_TIME	120 s	initiator rekeys when the session is this old and traffic is pending
REKEY_AFTER_MESSAGES	2⁶⁰ (1ULL << 60)	rekey after this many messages sent on a key
REJECT_AFTER_TIME	180 s	drop packets / expire a session older than this
REJECT_AFTER_MESSAGES	U64_MAX – COUNTER_WINDOW_SIZE – 1 (≈ 2 ⁶⁴)	hard counter ceiling; force rekey before reaching it
REKEY_TIMEOUT	5 s (+ 0-333 ms jitter)	resend interval for an unanswered Initiation
REKEY_ATTEMPT_TIME	90 s	

Constant	Value	Meaning
		give up handshake retries after this; clear queued packets
MAX_TIMER_HANDSHAKES	90 / REKEY_TIMEOUT = 18	max Initiation resends before giving up (= REKEY_ATTEMPT_TIME/REKEY_TIMEOUT)
KEEPALIVE_TIMEOUT	10 s	passive keepalive after this much receive-idle (with data sent)
COOKIE_SECRET_MAX_AGE	2 × 60 = 120 s	cookie / responder secret rotation period (§5)
Session zeroization	after REJECT_AFTER_TIME × 3	all ephemeral + symmetric keys wiped

So the spec doc's read-only timer constants [v02 wireguard-core §3.6] are all correct — REKEY_AFTER_TIME = 120, REKEY_TIMEOUT = 5, REKEY_ATTEMPT_TIME = 90, REJECT_AFTER_TIME = 180, KEEPALIVE_TIMEOUT = 10, REKEY_AFTER_MESSAGES = 2⁶⁰.

Replay window: WireGuard uses a sliding window keyed on the greatest counter seen, "a window of roughly 2000 prior values, checked after verifying the authentication tag" [wg-protocol]; the header sets COUNTER_BITS_TOTAL = 2048 [wg-messages-h]. This CONFIRMS the spec doc's UNVERIFIED: "2048-bit / 8000-packet-class window" — it is a **2048-entry** window [v02 wireguard-core §4.2].

Roaming: the Transport Data header carries no source address — WireGuard latches a peer's endpoint to whatever source a *cryptographically valid* packet arrives from, which is what makes Wi-Fi↔cellular handoff seamless and is safe by construction (a spoofed source cannot move the endpoint, since it cannot forge the AEAD tag) [v02 wireguard-core §8.4; mechanism consistent with wg-protocol's counter+AEAD authentication]. UNVERIFIED: the protocol page does not state the endpoint-latch rule verbatim; it is documented behaviour of the reference implementation and the spec doc's [01-DP §8.2].

10. boringtun — Cloudflare's userspace Rust WireGuard (CONFIRMED API)

What it is: "an implementation of the WireGuard® protocol designed for portability and speed" — a pure-Rust userspace WireGuard by Cloudflare [boringtun-gh].

Maintenance status (honest, §11.4.6): the README warns "Boringtun is currently undergoing a restructuring. You should probably not rely on or link to the master branch right now" and directs users to the **crates.io** releases instead [boringtun-gh]. So: actively published on crates.io, but the **master branch is explicitly not stable** — `helix-wg` MUST pin a crates.io version, never track master (this directly validates the spec doc's "pin to the exact crate version" caveat [v02 wireguard-core note 2]).

Crates shipped [boringtun-gh]:

- `boringtun` — the library, "for implementing WireGuard clients on various platforms (iOS, Android, etc.)" — this is what `helix-wg` embeds.
- `boringtun-cli` — "a userspace WireGuard implementation for Linux and macOS".

Production use: "successfully deployed on millions of iOS and Android consumer devices as well as thousands of Cloudflare Linux servers," including the Cloudflare 1.1.1.1 / WARP app [boringtun-gh]. This is the empirical grounding for choosing boringtun as HelixVPN's iOS/Android/cross-platform floor [v02 wireguard-core §1.1].

Platforms: x86_64 / aarch64 / armv7 Linux, macOS (x86_64 + Apple Silicon), iOS (multiple arches), Android [boringtun-gh].

License: 3-Clause BSD [boringtun-gh].

The Tunn sans-IO API (CONFIRMED signatures from docs.rs). Tunn "represents a point-to-point WireGuard connection" [boringtun-noise]. Exact signatures [boringtun-tunn]:

```
pub fn new(
    static_private: StaticSecret,
    peer_static_public: PublicKey,
    preshared_key: Option<[u8; 32]>, // ← the PSK slot, §6
    persistent_keepalive: Option<u16>,
    index: u32,
    rate_limiter:
        Option<Arc<RateLimiter>>, // ← cookie/anti-DoS, §5
) -> Self

pub fn encapsulate<'a>(&mut self, src: &[u8], dst: &'a mut [u8])
    -> TunnResult<'a>
```

```

pub fn decapsulate<'a>(&mut self, src_addr: Option<IpAddr>,
    datagram: &[u8],
    dst: &'a mut [u8]) -> TunnResult<'a>
pub fn update_timers<'a>(&mut self, dst: &'a mut [u8]) ->
    TunnResult<'a>
pub fn format_handshake_initiation<'a>(&mut self, dst: &'a mut
    [u8],
    force_resend: bool) ->
    TunnResult<'a>
pub fn set_static_private(&mut self, static_private: StaticSecret,
    static_public: PublicKey,
    rate_limiter: Option<Arc<RateLimiter>>)
pub fn stats(&self) -> (Option<Duration>, usize, usize, f32,
    Option<u32>)

```

TunnResult variants (CONFIRMED): Done, Err, WriteToNetwork, WriteToTunnelV4, WriteToTunnelV6 [boringtun-noise]. The noise module also exposes HandshakeInit, HandshakeResponse, PacketCookieReply, PacketData, and submodules errors, handshake, rate_limiter [boringtun-noise].

This **CONFIRMS** the spec doc's WgVerdict mapping and the four-verdict model [v02 wireguard-core §2.1] — the previously-UNVERIFIED: boringtun surface (note 2) is now grounded:

- Tunn::encapsulate(src, dst) ↔ spec encapsulate(peer, ip_pkt, scratch);
- decapsulate(src_addr, datagram, dst) ↔ spec handle_transport_in (note the src_addr: Option<IpAddr> arg — boringtun itself takes the source for roaming);
- update_timers(dst) ↔ spec tick;
- the caller-provided dst: &'a mut [u8] scratch buffer is exactly the spec's zero-alloc-hot-path invariant W1 [v02 wireguard-core §0.1].

UNVERIFIED: whether the *latest* crates.io release keeps these exact signatures — docs.rs "latest" was read 2026-06-25; pin + re-confirm at the chosen version [boringtun-tunn].

11. The four backends — boringtun vs kernel WG vs wireguard-go vs wireguard-nt

Backend	What	Where it fits	Source
kernel WireGuard	in-tree Linux kernel module (mainline since	HelixVPN Linux default for plain-udp; cannot emit	[wg-protocol, wg-

Backend	What	Where it fits	Source
	Linux 5.6); fastest path	obfuscated framing	wikipedia], [v02 §1.2]
boringtun	Cloudflare userspace Rust; sans-IO Tunn	HelixVPN cross- platform floor: iOS/Android/ macOS + Linux fallback	[boringtun- gh]
wireguard- go	the official userspace Go reference implementation; "quite functional"	reference + the historical Android/ Windows userspace path; Mullvad's DAITA forks it	[wg- xplatform, research- mullvad §2]
wireguard- nt	"High performance in- kernel WireGuard implementation for Windows " (NDIS)	the official + recommended Windows data path; Win10/11 on AMD64/x86/ ARM64; ships as wireguard.dll	[wg-nt]

Per-platform tradeoffs (grounded):

- **Linux:** kernel WG is the fast path; boringtun is the userspace fallback when the module is unavailable or the process is container-constrained [v02 §1.2]. Any **obfuscating** transport (MASQUE/Shadowsocks/LWO) forces userspace because the kernel cannot emit the carrier framing [v02 §9.3] — a first-class HelixVPN edge case.
- **iOS / macOS (NE):** there is **no kernel WireGuard on iOS**, so boringtun (or wireguard-go) in the NEPacketTunnelProvider is the *only* option; boringtun's bounded-memory Rust + millions-of-devices track record is the make-or-break gate [boringtun-gh, v02 §1.2].
- **Android:** userspace (boringtun or wireguard-go via VpnService+JNI); no root needed [v02 §1.2; wireguard-go is the Mullvad DAITA base, research-mullvad §1].
- **Windows:** wireguard-nt is the modern **in-kernel** path and "the only official and recommended way of using WireGuard on Windows"; it ships as an embeddable wireguard.dll [wg-nt]. The earlier Windows path was wireguard-go userspace over **Wintun** (the userspace TUN driver). HelixVPN can run boringtun over Wintun OR drive wireguard-nt's kernel data path — UNVERIFIED: which HelixVPN picks; settle in the Windows shim spec [v02 §1.2].

wireguard-go relevance to HelixVPN (cross-ref): Mullvad's DAITA traffic-shaping is implemented as a **wireguard-go fork** that reports per-packet send/recv events into the maybenot engine [research-mullvad §1-2]. If HelixVPN's DAITA layer (helix-daita) mirrors that, the Go data path is the proven integration point — but HelixVPN's Rust-first stack would instead wire maybenot (also Rust) against boringtun's Tunn event surface [research-mullvad §2]. UNVERIFIED: boringtun does not expose a documented per-packet maybenot hook today; that is HelixVPN integration work, not an off-the-shelf boringtun feature.

12. Known limitations of bare WireGuard → why HelixVPN wraps it

WireGuard is deliberately minimal; several gaps are exactly what HelixVPN's surrounding layers exist to close [wg-protocol, research-mullvad, v02]:

1. **No built-in traffic obfuscation.** WireGuard packets have a recognizable structure (fixed message types, sizes) and run over plain UDP — trivially DPI-fingerprinted and UDP-blockable. WireGuard has **no anti-censorship transport of its own** → HelixVPN's helix-transport ladder (MASQUE/H3, Shadowsocks, UDP-over-TCP, LWO) wraps the WG datagrams so a censor sees ordinary HTTPS/TCP, not WG [research-mullvad §5, v02 §9.2].
2. **Size/timing side-channels.** Even encrypted, WG packet **sizes and timing** leak to ML website-fingerprinting → DAITA/maybenot padding + cover traffic [research-mullvad §1-2]. WireGuard itself does nothing here.
3. **Rekey every ~2 minutes + handshake-on-first-packet.** REKEY_AFTER_TIME = 120 s means a session rekeys roughly every 2 minutes [wg-messages-h]; a cold tunnel must complete a 1-RTT handshake before the first data packet flows (adds latency on connect/roam). helix-wg budgets for this (handshake < 1 s plain, < 2 s MASQUE) [v02 §15].
4. **Roaming is connectivity-only, not anti-correlation.** Endpoint latching makes roaming seamless but a single server still sees source↔destination → HelixVPN multi-hop (WG-in-WG) splits that trust [research-mullvad §3, v02 §9.4].
5. **No post-quantum protection by default.** X25519 is harvest-now-decrypt-later vulnerable; the PSK slot (§6) is the *mechanism* but bare WG ships it empty → HelixVPN's additive PQ-PSK layer fills it [research-mullvad §4, v02 §4.4].
6. **No identity/PKI/coordination.** WireGuard is just keys + AllowedIPs; it has no enrollment, revocation, or route distribution → HelixVPN's control plane (coordinator +

RouteMap) owns that; helix-wg only executes upsert_peer / remove_peer [v02 §6.2–§6.3].

The honest framing (§11.4.6): WireGuard is a *correct, minimal, fast* crypto core — its "limitations" are scope boundaries, not defects. HelixVPN's value is the wrapping (obfuscation, shaping, PQ, coordination), with WireGuard as the unmodified, well-audited inner tunnel.

13. What this dossier confirms for helix-wg (closing the UNVERIFIED markers)

Spec-doc UNVERIFIED: (v02 wireguard-core)	Status now	Source
Msg sizes 148 / 92 / 64 / 16-hdr (§3)	CONFIRMED	[wg-protocol]
Curve25519 / ChaCha20Poly1305 / BLAKE2s + SipHash24 + XChaCha20Poly1305 (§4.1)	CONFIRMED	[wg-protocol]
Key-schedule chaining (§4.2)	CONFIRMED (HMAC chain)	[wg-protocol]
Replay window "2048-bit" (§4.2)	CONFIRMED = 2048-entry sliding window	[wg-protocol, wg-messages-h]
Timer constants 120/5/90/180/10 s, 2 ⁶⁰ msgs (§3.6)	CONFIRMED	[wg-messages-h]
MTU 1420 = 1500 – 60(v4)/80(v6) overhead (§7)	CONFIRMED	[wg-mtu]
boringtun Tunn encapsulate/decapsulate/ update_timers + TunnResult (§2, note 2)	CONFIRMED signatures	[boringtun-tunn, boringtun-noise]
boringtun maintenance status (§1.1)	CONFIRMED: master unstable, use crates.io	[boringtun-gh]
PSK = post-quantum injection point (§4.4)	CONFIRMED mechanism (PSK→symmetric schedule)	[wg-protocol, research-mullvad §4]

Spec-doc UNVERIFIED: (v02 wireguard-core)	Status now	Source
Cookie TTL / rate-limit (§14.3)	CONFIRMED: cookie secret 120 s	[wg- messages-h]

Still UNVERIFIED: and tracked for implementation-time pinning: (a) exact PQ-PSK KEM message framing (Mullvad-specific, §6); (b) Windows kernel-vs-userspace split (§11); (c) whether the *pinned* boringtun release keeps today's Tunn signatures (§10); (d) the boringtun↔maybenot per-packet hook (§11); (e) the endpoint-latch roaming rule verbatim from an official spec page (§9).

Honest gaps / unreachable sources (§11.4.6 / §11.4.99)

- [wireguard.com/papers/wireguard.pdf](https://www.wireguard.com/papers/wireguard.pdf) — **UNREACHABLE via the fetch tool**: it returned raw FlateDecode-compressed PDF binary, not parseable text [wg-paper-UNREACHABLE]. Every value the whitepaper canonically sources was re-confirmed from the official protocol page [wg-protocol] and the official constants header [wg-messages-h], so no claim above rests on the unreachable PDF. A human SHOULD still read the PDF directly to confirm the formal key-schedule proof and the Noise_IKpsk2 security argument before relying on §4/§6.
- The exact wording of WireGuard's **endpoint-roaming latch** rule was not found verbatim on the fetched official pages; it is documented reference-implementation behaviour [v02 §8.2] and consistent with the AEAD-authenticated counter design [wg-protocol] — labelled UNVERIFIED: in §9.
- boringtun's **per-packet event hook for traffic-shaping** (maybenot) is not a documented public API in the fetched README/docs — treated as HelixVPN integration work, not an existing boringtun feature (§11).

Sources verified

- wg-protocol — <https://www.wireguard.com/protocol/> — accessed 2026-06-25 (REACHED, primary)
- wg-paper-UNREACHABLE — <https://www.wireguard.com/papers/wireguard.pdf> — accessed 2026-06-25 (**UNREACHABLE**: returned FlateDecode binary, not parseable text)
- wg-xplatform — <https://www.wireguard.com/xplatform/> — accessed 2026-06-25 (REACHED)

- wg-messages-h — <https://github.com/WireGuard/wireguard-monolithic-historical/blob/master/src/messages.h> — accessed 2026-06-25 (REACHED, constants)
- boringtun-gh — <https://github.com/cloudflare/boringtun> — accessed 2026-06-25 (REACHED)
- boringtun-noise — <https://docs.rs/boringtun/latest/boringtun/noise/index.html> — accessed 2026-06-25 (REACHED, partial)
- boringtun-tunn — <https://docs.rs/boringtun/latest/boringtun/noise/struct.Tunn.html> — accessed 2026-06-25 (REACHED, full signatures)
- wg-nt — <https://git.zx2c4.com/wireguard-nt/about/> — accessed 2026-06-25 (REACHED)
- wg-mtu — WireGuard MTU overhead (1500 – 60 IPv4 / – 80 IPv6 = 1420), corroborated by GL.iNet/Calico/pfSense/procustodibus + en.wikipedia.org/wiki/WireGuard — accessed 2026-06-25 (REACHED, multi-source web search)
- wg-timers-search — WireGuard timer constants (REKEY_AFTER_TIME 120 / REKEY_TIMEOUT 5 / REKEY_ATTEMPT_TIME 90 / REJECT_AFTER_TIME 180 / KEYPALIVE_TIMEOUT 10), corroborated by [wireguard-go device/timers.go](https://wireguard-go.dev/device/timers.go) + [torvalds/linux drivers/net/wireguard/timers.c](https://torvalds/linux/drivers/net/wireguard/timers.c) — accessed 2026-06-25 (REACHED, web search)
- wg-wikipedia — <https://en.wikipedia.org/wiki/WireGuard> (kernel mainline since Linux 5.6) — accessed 2026-06-25 (REACHED, via search)
- research-mullvad — v09-research/research-mullvad.md (sibling dossier: PQ-PSK §4, obfuscation §5, multihop §3, DAITA/maybenot §1-2) — local evidence base
- v02 wireguard-core — v02-data-plane/wireguard-core.md (the spec doc this dossier backs) — local